

Recoverability-Gated Deliberation for Actionable Slow Reasoning in Closed-Loop Driving

Anonymous Authors

Abstract—Large language models can provide deliberative reasoning for autonomous driving, but inference latency separates the state that issues a request from the state in which its response may be applied. Existing asynchronous architectures preserve vehicle control during inference but leave the release decision unresolved. This paper presents Recoverability-Gated Deliberation (RGD), a two-stage framework for slow-reasoner admission and response release. RGD treats a delayed response as an expiring control proposal and distinguishes the opportunity remaining in the release state from the utility of the returned action. The admission gate jointly evaluates delay feasibility, action support, recovery-cost advantage, and scene demand before issuing a slow request. At response time, release validation remaps the proposal to the observed state and authorizes it only when the mapped action remains feasible, differs from the fast action, and has a lower recovery cost. Recoverable Value of Deliberation (R-VoD) provides an offline matched-rollout diagnostic of the corrective opportunity retained at release. Controlled experiments on highway-env and MetaDrive show that RGD selectively issues slow-path requests under the admission contract, preserves fast-path fallback for rejected responses, and operates across the evaluated delays and LLM backends. Cross-simulator experiments support portability of the shared action interface. Response-level traces confirm that only proposals satisfying all release checks can change the final projected command.

Index Terms—Autonomous driving, closed-loop decision making, inference latency, large language models, recoverability, selective computation.

I. INTRODUCTION

LARGE language models (LLMs) and vision-language models (VLMs) have introduced semantic scene interpretation, driving knowledge retrieval, and high-level action selection into autonomous driving systems [1]–[8]. These capabilities augment reactive policies with world knowledge and long-horizon intent. However, closed-loop deployment exposes a timing problem that many existing designs do not explicitly address: a reasoning request is issued from one traffic observation, whereas the returned proposal may be applied in a later, different state. A high-frequency controller maintains continuous vehicle control while the deliberative model reasons asynchronously. The control cadence is preserved, but the system must still determine whether the delayed proposal should be applied when it returns.

A concrete scenario illustrates the gap. A closing lead vehicle and an initially open adjacent lane may trigger a lane-change request to the slow reasoner. During inference, surrounding vehicles advance, the fast controller continues to act, and the target gap may narrow or close. By the time the response arrives, the original maneuver may have expired, become physically infeasible, or lost its advantage over the fast controller’s current

action. The system must therefore determine whether the opportunity persists, whether the proposed action differs from the fast action, and whether it has a lower recovery cost in the observed state.

Maintaining a fast control stream does not by itself resolve the release decision. A returned action may remain feasible but be equivalent to the fast action, or it may be less suitable after the surrounding traffic changes. Checking the slow proposal in isolation is therefore insufficient. Both actions must use the same action schema and mapper and be evaluated in the same current state. This requirement motivates the separation between query admission and release validation in RGD.

Recent work addresses parts of this process. Invocation policies use uncertainty, risk, scene complexity, or policy discrepancy to decide when to engage the slow branch [9]–[11]. VLM-based attention allocation prioritizes surrounding vehicles under behavioral uncertainty [12]. AsyncDriver and NetRoller preserve fast control during model inference [8], [13], while DualDrive aligns semantic intent with a real-time policy [14]. Runtime assurance verifies whether a candidate action may enter the actuation path. These methods address invocation timing, control continuity, or safety checking. However, they do not explicitly compare the delayed proposal with the contemporaneous fast action at response time using the same feasibility test.

This paper addresses two questions in delayed closed-loop driving: whether the evolved traffic state still admits a corrective action and whether the returned proposal provides such an action at release. Two constraints guide the design: the fast controller must remain active throughout inference, and rejected proposals must not disrupt the control loop. Both proposals are evaluated at the same release state through a shared action mapper and feasibility interface.

RGD treats the slow reasoner as an optional proposal generator rather than a replacement for the fast controller. The fast action remains the default, and a slow proposal reaches actuation only through the shared mapper and safety interface. This separation allows the reasoner to be changed without modifying the release rule and makes the source of each executed action explicit.

A release state is *recoverable* when the remaining maneuver window contains at least one feasible action that differs from the fast action and exceeds a matched-rollout return margin. At query time, RGD applies online delay, support, cost, and demand conditions before issuing a slow request. *Actionability* is evaluated at release: the returned proposal must be feasible, differ from the fast action, and have a lower recovery cost under the shared action evaluator. Recoverability therefore describes

the release state, whereas actionability describes the returned proposal.

Recoverable Value of Deliberation (R-VoD) quantifies recoverability offline. It compares alternative and fast first actions through matched rollouts from the same logged release state under a common continuation policy. It therefore measures the corrective opportunity remaining after the response interval independently of the returned slow response. R-VoD is a simulator-based diagnostic rather than an online value estimate or admission target. Gate parameters are selected on a separate development cohort. R-VoD does not use the returned model output and is excluded from the online rule.

The online gate requires scene demand, delay feasibility, action support, and cost advantage simultaneously; high urgency alone cannot compensate for an expired maneuver window or the absence of a supported alternative. The fast controller supplies every action during inference. When the response arrives, release validation evaluates the proposal in the current traffic state and accepts it only if it passes current-state mapping, state, distinctness, and cost-advantage checks.

We evaluate RGD by comparing six allocation policies under a shared fast controller, slow reasoner, request budget, and feasibility interface across multiple delay conditions. Event-level response traces connect each request to release-state validation and final actuation, while component ablations examine how each admission condition changes the selected release states. Further experiments replace the LLM backend and simulator adapter while preserving the action schema and fallback path. Auxiliary traffic sweeps describe operation under the fixed interface.

The main contributions are as follows:

- 1) We formulate delayed deliberation as an expiring corrective opportunity and distinguish release-state recoverability from response-specific actionability. R-VoD measures the remaining opportunity through matched first-action rollouts under a common continuation policy, independently of the language model that returns a proposal.
- 2) We present RGD, which combines noncompensatory query admission with an action-aligned release contract. The resulting non-interference property keeps the fast controller active throughout inference: a delayed proposal can alter control only after both branches are mapped and evaluated in the observed release state through the same safety interface.
- 3) We provide an evidence suite spanning controlled-delay allocation, response-to-actuation traces, component analysis, reasoner replacement, and cross-simulator evaluation. The studies separately test selective invocation, release-contract enforcement, and portability of the shared action interface.

II. RELATED WORK

A. Language-Conditioned and Generative Driving

Language-conditioned driving methods differ in the proximity of their outputs to vehicle actuation. At the advisory level, DiLu retrieves relevant experience before producing an

interpretable decision [1], and Driving with LLMs represents traffic through object-level vectors and returns actions with explanations [15]. VLM-based systems combine visual observations with language reasoning for navigation and motion planning [6], [16], while knowledge retrieval methods supply domain-specific context before decision making [7]. These interfaces use language reasoning to encode scene relations and driving knowledge that fixed rules cannot represent.

A second group places model outputs closer to the control boundary. Policy selection systems choose an executable behavior from a predefined library [17], GPT-Driver formulates trajectory planning as language modeling [5], and C-TRAIL couples commonsense recall with trust-aware trajectory search [18]. Generative driving models learn predictive scene representations or multimodal future trajectories to guide downstream planning [19]–[22]. As model outputs move closer to actuation, fewer downstream modules remain available to compensate for stale proposals, and the output must be validated against the state in which it will execute.

These studies primarily improve model predictions or driving-knowledge representations. RGD addresses a different system decision: whether a delayed high-level action may replace the current fast action. It does not require a new language model or trajectory decoder and operates through the common action interface used by existing slow reasoners. As outputs move closer to actuation, inference-to-execution latency becomes more consequential. RGD therefore evaluates the returned proposal in the state in which it may be executed.

B. Dual Process Reasoning and Selective Computation

Dual-process driving systems divide responsibility between a responsive policy and a deliberative model. LeapAD uses reflection and accumulated experience, LeapVAD adds cognitive perception, and FASIONAD coordinates both branches through adaptive feedback [9], [23], [24]. DualDrive aligns semantic intent with a real-time policy, and NetRoller connects general and specialized models through asynchronous execution [8], [14]. In each design, the fast branch supplies immediate control while the slow branch contributes information or actions at selected intervals.

Selective computation methods determine when the slow branch should run. AdaDrive and AdaThinkDrive adapt deliberation depth to scene demand [10], [25], and G-VLM calibrates invocation from the discrepancy between lightweight and multimodal policies [11]. Behavioral Attention uses a VLM to allocate attention among surrounding vehicles under behavioral uncertainty [12]. These invocation signals are evaluated before inference begins, when neither the returned action nor the release state is known. RGD supplements invocation selection with release-time evaluation against the contemporaneous fast action.

Query-time selection and release-time validation serve different purposes. The first determines whether slow computation should begin; the second determines whether its output remains useful after the state has changed. RGD retains both decisions explicitly instead of applying a response solely because the original request was admitted.

Metareasoning balances expected decision improvement against computational cost. Time-bounded and anytime methods schedule reasoning under a finite budget [26]–[29], and concurrent planning permits an executor to continue acting while a planner computes [30]. In driving, invocation value at execution may differ from its query-time estimate because inference consumes the maneuver window while interim fast actions reshape the decision context. R-VoD captures this phenomenon by measuring the corrective opportunity remaining at release through matched rollouts.

C. Runtime Safety, Delay, and Viability

Timing mechanisms preserve control during inference. AsyncDriver separates language-model planning from real-time control [13], NetRoller keeps a specialized driving model active [8], and event-triggered or adaptive-computing strategies regulate update frequency [31], [32]. Delay-aware control further models state evolution during input lag [33]. At the assurance layer, Simplex-based systems revert to a trusted controller when a learned action fails an acceptance test [34], and formal viability methods characterize admissible motion under dynamic constraints [35], [36].

These approaches address what to infer, when to invoke, or whether an output is safe. They do not explicitly compare a delayed proposal with the contemporaneous fast action at release under a shared feasibility interface.

RGD complements rather than replaces runtime safety mechanisms. The safety interface determines whether an action can be applied, while release validation also compares the delayed proposal with the current fast action. A safe but equivalent proposal therefore does not replace the fast action, and the shared safety shield remains active after action selection.

III. RECOVERABILITY-GATED DELIBERATION

RGD separates a slow-reasoner call into query admission and release validation. Admission determines whether the scene warrants deliberation and whether a feasible alternative is supported under the expected delay. Release validation checks the returned proposal in the observed release state. The fast controller supplies the executed action while inference is pending and whenever either stage rejects the slow path.

A release state s_r is *recoverable* when it retains a feasible action whose matched-rollout return exceeds that of the contemporaneous fast action by at least ϵ . R-VoD measures this opportunity offline. A returned response is *actionable* when its current-state action is feasible, different from the fast action, and lower in recovery cost. Thus, the offline measure characterizes the available corrective opportunity, whereas release validation tests whether a particular response realizes it.

As shown in Fig. 1, the online path contains the two gates, whereas the offline branch constructs R-VoD labels from matched first-action rollouts. The fast controller remains active at every cycle. Driving memory can provide context after admission, but it does not alter either gate and is not used in the reported frozen runs.

The architecture imposes a non-interference contract on slow reasoning. A slow request never suspends the fast controller, and a missing response, mapping failure, or rejected release leaves the fast command selected. The reasoner is therefore a proposal source rather than a second controller. This boundary prevents inference latency and response-format errors from changing the control cadence.

The release comparison is action-aligned. Both fast and slow proposals first enter the same mapped command space, after which the same feasibility and cost interfaces are applied. Semantically equivalent language outputs are therefore treated as the same maneuver, and backend-specific wording cannot create an artificial action difference. The shared final safety shield remains the last stage for either source.

A. Query and Release Process

At frame t , the ego vehicle observes state s_t . The fast policy π_f and the shared mapper produce the mapped fast command a_t^f . Both branches use the same mapper, feasibility test, and final safety shield $\Phi(s, a)$, which maps command a to its final executable action in state s . The mapper converts each proposal into a command and target speed; different expressions are equivalent when they induce the same mapped behavior. The high-level action domain comprises lane-left, lane-hold, lane-right, accelerate, and decelerate, all mapped through the same simulator interface.

The returned proposal is mapped against the actions supported at release. A lane-change or speed command available at query time may no longer be valid in s_r ; a mapping or feasibility failure leaves a_r^f selected.

The delay estimate is used only for admission. The actual release frame r is the first control frame at which the response is available, and returned and fast actions are evaluated in the observed state s_r . Responses unavailable before episode termination are not released.

B. Offline Measure of Corrective Opportunity

Offline matched rollouts quantify the corrective opportunity at release. At a logged state s , let $a_{\text{eff}}^f = \Phi(s, a^f)$ denote the effective fast first action after the shared mapper and final safety projection. $\mathcal{D}(s)$ contains feasible effective first actions that remain distinct from a_{eff}^f after the same stages, and $\mathcal{C}(s) = \{a_{\text{eff}}^f\} \cup \mathcal{D}(s)$. The matched-rollout utility and Recoverable Value of Deliberation (R-VoD) are

$$J(s, a) = \frac{\sum_{k=0}^{K-1} \gamma^k \bar{r}_k(s, a)}{\sum_{k=0}^{K-1} \gamma^k} - \lambda_{\text{col}} \mathbb{I}_{\text{col}}(s, a), \quad (1)$$

$$\text{R-VoD}(s) = \max_{a \in \mathcal{C}(s)} [J(s, a) - J(s, a_{\text{eff}}^f)].$$

Here, K is the common positive rollout horizon, $\gamma \in (0, 1]$ is the discount factor, $\bar{r}_k$ is the normalized simulator reward, and $\lambda_{\text{col}} > 0$ weights the collision indicator $\mathbb{I}_{\text{col}}(s, a)$. Reward increments after termination are set to zero. Only the first effective action is changed; the fast policy supplies the continuation. Because $a_{\text{eff}}^f \in \mathcal{C}(s)$, R-VoD is nonnegative and equals zero when no alternative improves the matched fast

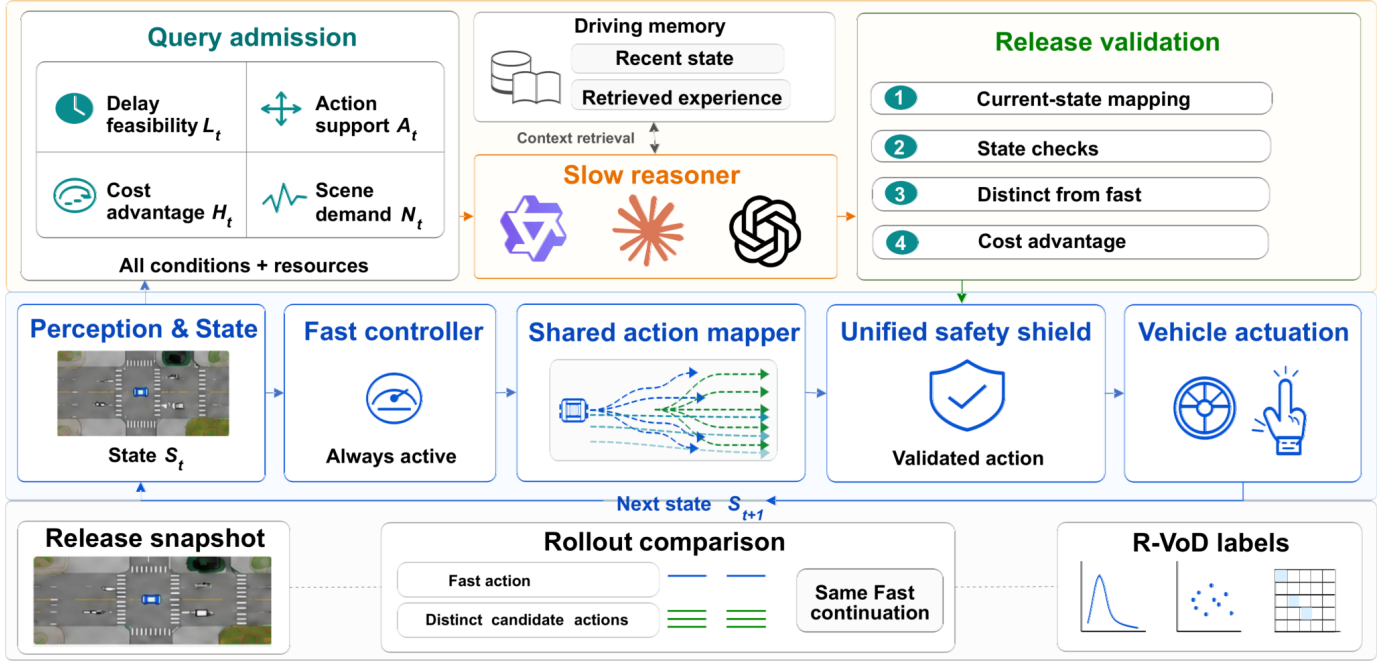


Fig. 1. RGD architecture and information flow. Query admission evaluates delay feasibility, action support, cost advantage, and scene demand. The optional driving-memory block supplies retrieved context and was disabled in the frozen evaluations. Release-validation stage 1 invokes the shared action mapper used by the fast path. The returned action is then validated before the unified safety shield. Matched rollouts from release snapshots provide the offline R-VoD labels.

continuation. A state is labeled corrective when $R\text{-VoD}(s) \geq \epsilon$, where $\epsilon > 0$ is a fixed diagnostic margin.

The matched design fixes the logged state, traffic realization, maximum horizon, continuation policy, and termination rule. The maximization in (1) is therefore over feasible first actions rather than LLM responses. R-VoD is computed after data collection and does not enter the online gate; the online cost below ranks candidate actions for admission and release.

Changing only the first effective action isolates the corrective opportunity available in the release state under the common simulator continuation. R-VoD therefore does not reward response length, explanation quality, or a particular LLM backend. It supplies a state-level label against which an admission policy can be evaluated, while release validation separately determines whether the actual returned proposal realizes that opportunity.

C. Query Admission

The online gate evaluates four conjunctive conditions. All must hold; high scene demand cannot compensate for an expired maneuver window.

Let $\mathcal{A}_t$ denote the current mapped action set. The fixed evaluator assigns each member a normalized recovery cost $c_t : \mathcal{A}_t \rightarrow [0, 1]$, where lower values are preferred, and a normalized support $u_t(a) \in [0, 1]$. Let $\mathcal{D}_t^{\text{alt}}$ contain feasible actions whose mapped commands differ from a_t^f . For each alternative action family m , its support $\bar{u}_{t,m}$ is the largest $u_t(a)$ among feasible members; an unavailable family contributes zero. Action support A_t is the mean of these family-wise maxima and is set to zero when no alternative family is available or the evidence is incomplete. Let $c_t^{\text{alt}} = \min_{a \in \mathcal{D}_t^{\text{alt}}} c_t(a)$ when the set is nonempty and $c_t^{\text{alt}} = c_t(a_t^f)$ otherwise.

The remaining gate factors are

$$\begin{aligned} L_t &= \text{clip} \left(1 - \frac{\hat{\ell}_t + \rho_t}{\max(d_t, \varepsilon_d)}, 0, 1 \right), \\ H_t &= \text{clip} \left(\frac{[c_t(a_t^f) - c_t^{\text{alt}}]_+}{\kappa}, 0, 1 \right), \\ N_t &= \max\{n_t^{\text{haz}}, n_t^{\text{int}}\}. \end{aligned} \quad (2)$$

Here, L_t is delay feasibility, H_t is cost advantage, and N_t is scene demand. The terms $n_t^{\text{haz}}, n_t^{\text{int}} \in [0, 1]$ are normalized hazard and interaction evidence, respectively. The timing quantities $d_t, \hat{\ell}_t, \rho_t \geq 0$, and $\varepsilon_d > 0$ use the same time unit; d_t is the estimated time to the earliest active maneuver constraint, ρ_t is a timing reserve, and ε_d is a timing regularizer. The positive constant κ normalizes the recovery-cost advantage. A bounded planning horizon replaces d_t when no finite constraint is active. The operator $[x]_+ = \max\{x, 0\}$, and $\text{clip}(x, 0, 1) = \min\{1, \max\{0, x\}\}$. If $\mathcal{D}_t^{\text{alt}}$ is empty, $H_t = 0$. The maximum in N_t avoids counting correlated hazard and interaction evidence twice.

The factors encode different necessary conditions. L_t tests whether computation fits within the maneuver window, A_t requires support across alternative action families, H_t requires at least one lower-cost alternative, and N_t establishes a reason to deliberate. Their conjunction is deliberately noncompensatory: high scene demand cannot offset expired timing, and broad action support cannot offset the absence of a cost advantage. This structure preserves the operational meaning of each gate factor across delay settings.

Resource availability, including the request budget, cooldown, and executor status, is represented by $v_t \in \{0, 1\}$; $v_t = 1$ also

requires that no slow request is pending. The admission rule is

$$q_t = \mathbb{I}[v_t = 1 \wedge L_t \geq \tau_L \wedge A_t \geq \tau_A \wedge H_t \geq \tau_H \wedge N_t \geq \tau_N], \quad (3)$$

where $\mathbb{I}[\cdot]$ is the indicator function and $\tau_j \in (0, 1]$, $j \in \{L, A, H, N\}$, are generic gate floors selected on development scenarios and fixed before evaluation.

If latency evidence, the action-cost vector, or the slow executor is unavailable, the corresponding condition is false and the fast controller continues without issuing a slow request.

D. Release Validation

At the actual release frame r , let y_r be the returned response and $\mathcal{M}$ the shared current-state mapper. The fast policy produces the mapped fast command a_r^f from the same state. If $\mathcal{M}(s_r, y_r)$ is undefined, the release gate is set to zero and the fast command is retained. Otherwise, let $\tilde{a}_r^{sl} = \mathcal{M}(s_r, y_r)$ denote the mapped slow command. Let $\mathcal{V}_r \subseteq \mathcal{A}_r$ denote the current feasible mapped commands when A_r , H_r , and N_r satisfy their release conditions, and let $\mathcal{V}_r = \emptyset$ otherwise. These quantities are recomputed at release; L_t is not. If $\tilde{a}_r^{sl} \notin \mathcal{V}_r$, the release gate is set to zero. Otherwise, the remaining test in Fig. 1 is

$$g_r = \mathbb{I}[\tilde{a}_r^{sl} \neq a_r^f \wedge c_r(a_r^f) - c_r(\tilde{a}_r^{sl}) \geq \tau_R]. \quad (4)$$

Here, $\tau_R \in (0, 1]$ is a generic release margin under $c_r(\cdot) \in [0, 1]$ and is fixed before evaluation.

The mapped slow command is selected when $g_r = 1$; otherwise, the fast command is retained. The shared safety projection Φ then produces the executed action. Distinctness is tested between mapped commands before Φ , so the shield may still map an authorized slow command to the same executed command as the projected fast action. The release-state H condition establishes that a lower-cost alternative exists, while the final term in (4) verifies that the returned proposal provides one. No R-VoD label enters this online decision.

Release validation is stricter than a slow-action safety test. Feasibility asks whether the mapped proposal can execute in s_r , distinctness asks whether it changes the contemporaneous fast behavior, and the final cost comparison asks whether that change improves the recovery objective. A proposal admitted at query time receives no persistent authority; all response-specific checks use the observed release state.

IV. EXPERIMENTS AND DISCUSSION

A. Experimental Setup

We evaluate RGD on the highway-env [37] and MetaDrive [38] closed-loop simulators with Qwen3-8B [39] as the default slow reasoner. The main highway-env experiments operate at 10 Hz for 30 s. Each comparison shares a fixed seed cohort, action schema, fast controller, and evaluation rule; gate development, main evaluation, and mechanism analysis use separate cohorts. The main analyses use 30 seeds; the component analysis uses a separate 20-seed cohort. Unless stated otherwise, intervals denote 95% percentile bootstrap

confidence intervals with simulator seed as the resampling unit. R-VoD is computed only after release through matched first-action rollouts. Gate floors, normalization constants, and corrective margins are selected on the development cohort and remain fixed throughout the reported evaluation.

The experiment suite gives each analysis a distinct evidentiary role. System-level context places the closed-loop implementation in established task settings. The delay sweep tests whether admission allocates slow computation according to the available maneuver window, while the response trace tests the release contract after the traffic state evolves. The component study examines how the joint gate changes the states selected for deliberation. Reasoner replacement tests the action interface independently of a particular model, and the traffic and simulator studies test whether the fixed interface operates across road structures and simulator adapters. This organization links each result to a specific system property.

B. System-Level Context

Following the PADriver experimental conditions [40], we evaluate RGD and place the resulting values alongside those reported in the source paper. The published rows and bars retain their source definitions and evaluation scope and provide task-level context rather than a statistical head-to-head comparison. Table I shows that RGD completes 29 of 30 runs, records 611 m and 73.60 km/h, and operates at 37.6 ms per frame without a dedicated training phase on the evaluation environment. The fast controller remains active at every frame, whereas the slow reasoner is invoked selectively. Saf. and Kep. denote safe-distance keeping and lane keeping, respectively.

Hu et al. [6] define success as 30 consecutive collision-free frames. We adopt their three density settings and ten seeds per setting with no added delay. Fig. 2 reports success rates alongside published values, and Table II provides RGD's operating metrics under the settings stated in the source paper.

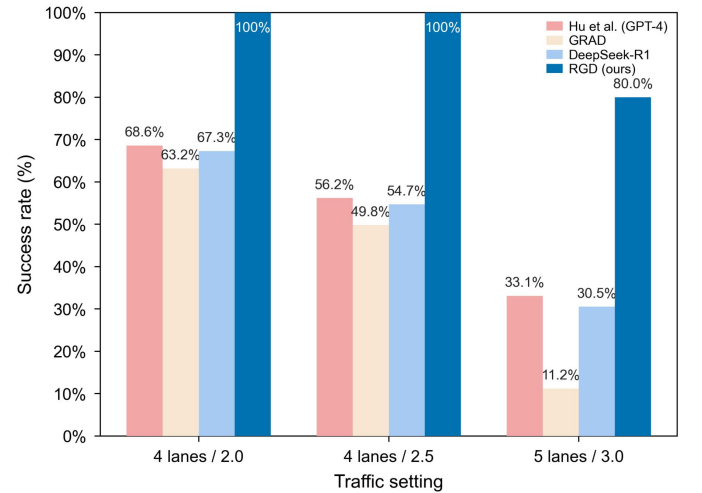


Fig. 2. Success-rate context in the highway-env settings of Hu et al. GPT-4, GRAD, and DeepSeek-R1 are source-reported values; their placement with RGD is not a head-to-head statistical comparison.

Under the settings stated by Hu et al., RGD attains 100% success at 4L/2.0 and 4L/2.5. At 5L/3.0, it attains 80% success

TABLE I
TASK-LEVEL CONTEXT IN THE PADRIVER SETTING.

Method	Distance (m)	Speed (km/h)	Saf.	Kep.	Completed	Runtime (s/frame)
Driving with LLMs	411	49.42	0.49	0.56	20	48.0
GPT-Driver	428	51.40	0.45	0.52	23	36.2
DiLu	386	46.29	0.60	0.72	28	24.1
PADriver-Normal	603	72.47	0.91	0.92	25	1.4
RGD (ours)	611	73.60	0.75	0.86	29	0.0376

TABLE II
RGD OPERATING METRICS IN THE SETTINGS OF HU ET AL.

Setting	Dist. (m)	Speed (m/s)	Requests/ep.	Runtime (ms/frame)
4L / 2.0	69.29	23.84	1.0	58.9
4L / 2.5	66.19	22.76	1.0	65.2
5L / 3.0	52.38	17.38	0.8	50.1

and schedules 0.8 slow-path requests per episode. Table II uses safety-qualified speed, assigns zero to unsuccessful episodes, and averages requests over the ten episodes in each setting. The per-frame runtime remains below 66 ms in all three settings.

C. Delay-Controlled Allocation and Component Evidence

1) *Zero Added Delay Reference*: The zero-delay experiment provides a reference in which all six allocation policies share the same fast controller, slow reasoner, request budget, and feasibility interface; only the request and release rules differ. The subsequent delay sweep uses the same seeds to examine how delay feasibility changes slow-path allocation.

Without added delay, Fig. 3(a)–(b) shows that RGD schedules fewer slow-path requests and has a lower per-frame runtime than Always Slow. This result shows that query admission reduces continual slow-path use under the same fast controller and slow reasoner.

Across the delay sweep in Fig. 3(c), the distance confidence intervals overlap, whereas the request count decreases at longer added delays. This allocation pattern is consistent with the delay-feasibility condition becoming more selective as the response interval grows. The fixed gate therefore changes its invocation rate without per-delay retuning.

2) *Allocation With a 1.7-s Added Delay*: The 1.7-s experiment examines how RGD processes a slow response after the traffic state has evolved. The fast controller, slow reasoner, request budget, seed cohort, and feasibility interface remain fixed. Fig. 4 reports the response-validation stages and the resulting release decisions.

Table III reports a separate 30-seed backend evaluation under the 1.7-s setting. “Calls” denotes successful service responses rather than request attempts in the main cohort.

The fixed delay ensures that each returned proposal is evaluated in a state later than the state that initiated the request. RGD therefore applies a returned proposal only after all release checks.

3) *Response Validation and Component Effects*: Response traces follow each request from admission through response validation, current-state mapping, and actuation. At the same 1.7-s added delay, the leave-one-out variants use a separate

TABLE III
RESULTS WITH DIFFERENT LLM BACKENDS AT A 1.7-S ADDED DELAY.

Reasoner	Calls	Distance (m)	Completion
Qwen3-8B	138	599.16	0.97
Qwen3.5-4B	138	599.16	0.97
Qwen2.5-7B	122	599.12	0.97
GPT-5.6-sol	112	603.39	0.97
GPT-5.6-terra	132	599.16	0.97
GPT-5.6-luna	133	599.16	0.97
Fable-5	137	599.16	0.97

20-seed cohort; each removes one admission condition while retaining the release procedure.

Under the 1.7-s delay, all 143 RGD requests return valid responses, and 138 are evaluated before episode termination. Current-state validation admits 28 responses to the release comparison; nine mapped slow actions differ from the contemporaneous fast action, and six pass the recovery-cost margin. After the shared final projection Φ , four of these commands match the projected fast command and two remain distinct. Fig. 4(a) reports the validation stages, while panel (b) summarizes all 28 release outcomes. RGD applies a delayed slow action only after all release checks; otherwise, the fast action remains in control.

Fig. 5(a)–(b) shows that removing L , A , or H changes both the number of requests and the release states selected by the gate. Panel (c) reports the corrective-state-rate difference between Full RGD and each ablation with 95% seed-bootstrap confidence intervals. The H contrast is the only one whose interval lies entirely above zero, giving the clearest association between an admission component and the corrective-state composition of the selected cohort. The selection changes in panels (a)–(b) further show that L and A shape which states are sent to the slow path. Together, these analyses show that the full rule combines delay, action availability, and recovery cost rather than relying on a single risk trigger.

The corrective-state rate in Fig. 5(c) is labeled offline by R-VoD, whereas H_t is computed online from the recovery-cost evaluator c_t . The two quantities serve different roles: H_t ranks current candidate actions before a request is issued, whereas R-VoD labels the opportunity that remains at the observed release state through matched rollouts. The positive H contrast therefore links an online admission signal to a separately constructed release-state diagnostic.

D. Reasoner-Interface Robustness

We replace only the slow reasoner while retaining the RGD gate, action schema, and fallback path. Table III reports similar

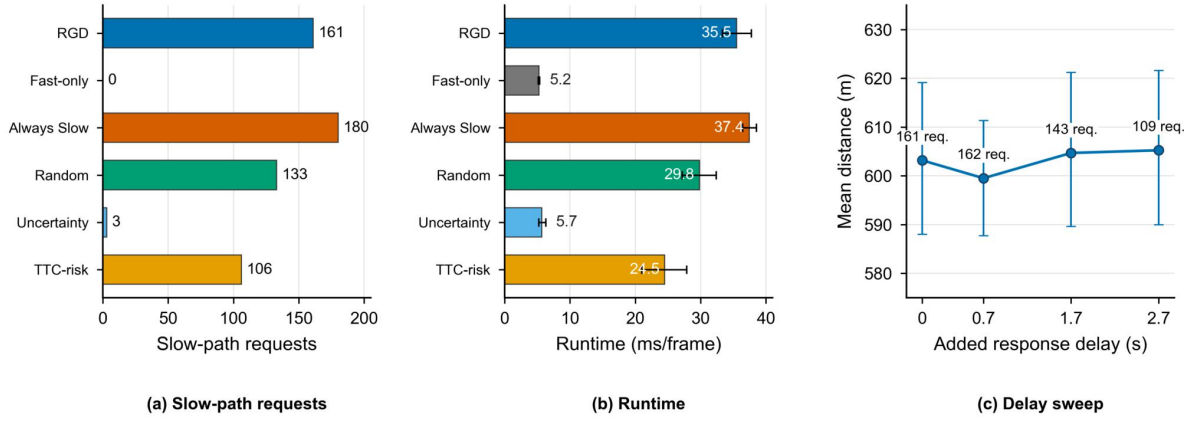


Fig. 3. Zero-delay policy comparison and RGD delay sweep: (a) slow-path requests, (b) runtime, and (c) distance and requests versus added delay. Request counts are totals across 30 seeds; error bars denote 95% seed-bootstrap confidence intervals for runtime and distance.

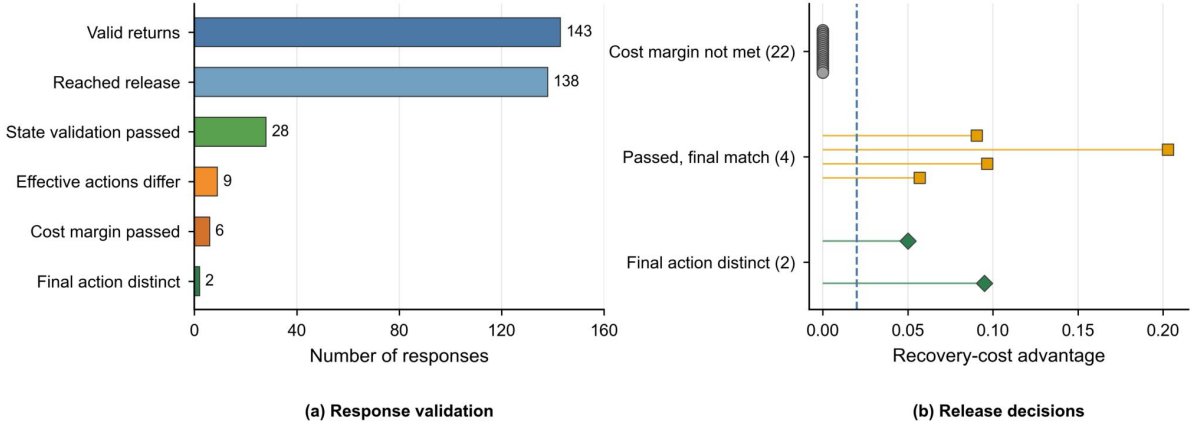


Fig. 4. RGD validation at a 1.7-s added delay: (a) responses after each validation stage and (b) recovery-cost advantages and final outcomes for 28 state-valid responses. Stage counts aggregate the 30-seed main cohort; the dashed line denotes the release margin τ_R .

distance and completion across all seven evaluated backends, from local models to API services. Successful service responses range from 112 to 138, whereas mean distance remains within 599–603 m and every backend attains a completion rate of 0.97. All backends use the same action mapping, current-state validation, and recovery-cost comparison. Under this shared interface, the system-level point estimates remain within a narrow observed range across the evaluated backends.

E. Evaluation Across Traffic and Simulators

1) *Traffic Structure*: The auxiliary sweep includes the default 4L/2.0 setting and five lane/density variants without retuning the gate. Table IV reports mean distance and successful runs over 30 episodes per setting. At density 2.0, successful runs range from 28 to 30 and mean distance from 599 to 629 m. Across all six cells, the fixed configuration completes 20–30 runs. These results describe operation of the same interface under the evaluated traffic structures.

2) *Simulator Transfer*: The MetaDrive experiment introduces simulator-specific observation and action adapters at zero response delay, with no modification to the gate configuration used in highway-env. The shared action schema, admission

rule, and fallback order are applied without retuning. This experiment evaluates interface portability across heterogeneous simulator dynamics; the delay-controlled release mechanism is assessed in the highway-env experiments.

Fig. 6 visualizes representative frozen cross-simulator traces. The upper row contains highway-env highway, merge, roundabout, and intersection states; the lower row contains the corresponding MetaDrive state reconstructions. The paired layouts expose the same discrete high-level action schema in straight-road, merging, circulating, and cross-traffic geometries. The scenes provide visual context for the quantitative transfer studies.

Road geometry, traffic representation, and simulator dynamics differ across the two rows, yet each panel enters the same mapper, admission rule, release contract, and fast-path fallback. The MetaDrive panels reconstruct the logged ego state and recorded task-relevant neighbors at the selected frame. This provides a direct view of the adapter boundary while preserving the traffic configuration used by the frozen evaluation.

F. Evaluation on Additional Scenario Families

The published ASR-RL and VLM-attention rows retain their source definitions and evaluation scopes, whereas the RGD

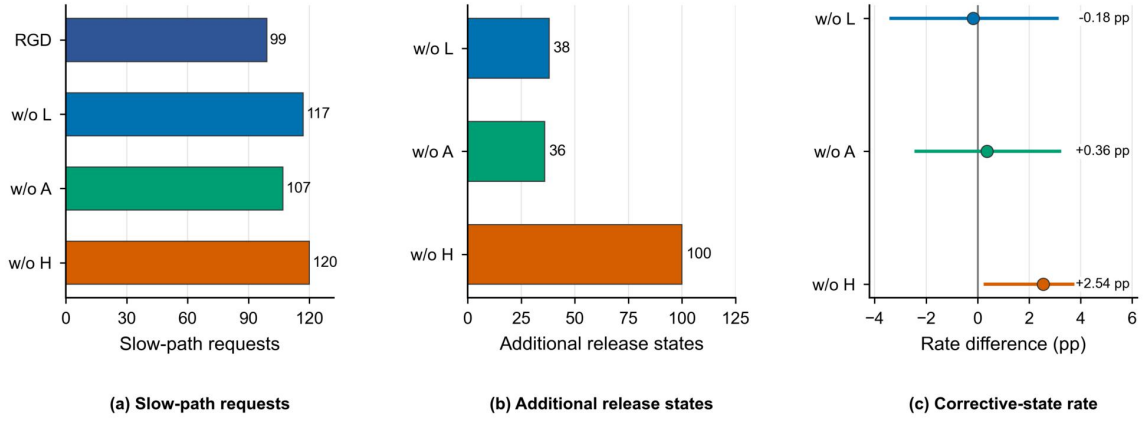


Fig. 5. RGD components: (a) slow-path requests, (b) states selected only by each ablation, and (c) corrective-state-rate differences computed as Full RGD minus ablation with 95% confidence intervals. Counts aggregate the separate 20-seed mechanism cohort.

TABLE IV
RGD OUTCOMES UNDER LANE AND DENSITY SETTINGS.

Lanes/Density	Distance (m)	Successful runs (/30)
4L/2.0	599	29
4L/3.0	478	23
5L/2.0	629	30
5L/3.0	429	20
6L/2.0	611	28
6L/3.0	453	20

TABLE V
COMPOUND-HAZARD TASK CONTEXT IN METADrive.

Method	Scene	Safety (%)	Speed (m/s)
ASR-RL [17]	A	99.2	7.1
	B	98.3	8.6
	C	97.9	6.8
RGD (ours)	A	100.0	4.8
	B	94.8	7.0
	C	84.6	6.9

TABLE VI
MANEUVER AND TRAFFIC-DENSITY TASK CONTEXT.

Method	Traffic Scenario					Left Turn Traffic Density		
	Left	Right	Straight	Merge	Round.	1.0	2.0	3.0
VLM attention allocation [12]	0.966	0.991	0.984	0.987	0.974	0.975	0.961	0.948
RGD (ours)	0.842	0.908	0.868	0.986	0.747	0.906	0.763	0.601

rows are measured in this work. The values provide task-level context rather than a common-protocol statistical ranking.

1) *Compound Hazard Scenes*: Following the experimental conditions stated by ASR-RL [17], the MetaDrive evaluation covers pedestrian crossing (A), uncontrolled intersection (B), and combined vehicle-pedestrian traffic (C), each with 1,000 episodes. Table V places the RGD results alongside the source-reported ASR-RL entries. RGD attains 100% collision-free safety in Scene A and operates across all three compound-hazard scenes without policy-library training or scenario-specific gate tuning.

2) *Maneuver and Density Changes*: Following the settings stated in the source paper, five maneuver classes and a left-turn density sweep are evaluated at 2 Hz for 30 s with 1,000 episodes per cell. Table VI reports the RGD success rates alongside the published VLM attention-allocation values [12] as task-level context. RGD achieves 0.986 on merge, and the same action interface and release rule remain operational across highway, junction, and curved-road maneuver families.

G. Discussion

RGD evaluates evidence at two control times. Before computation, admission asks whether a supported lower-cost

alternative can remain useful when the slow result is expected to return. After state evolution, release validation asks whether the returned proposal remains useful relative to the action the fast controller would now take. This separation distinguishes recoverability from actionability. It also distinguishes an expired opportunity, an unusable proposal, and final-shield equivalence, which otherwise appear identical if only the executed command is recorded.

The controlled-delay studies test the first decision, and the response trace tests the second. The delay sweep shows that the fixed admission rule changes slow-path allocation as the response interval changes. The trace then follows each proposal through current-state mapping, feasibility, distinctness, recovery-cost comparison, and the shared safety projection. Together, these analyses show that a successful response alone does not establish control authority and that the fast path remains the default whenever the contract is not satisfied.

The component analysis connects the admission rule to release-state evidence. The *H* contrast is the only corrective-state-rate interval lying entirely above zero, and R-VoD provides its separately constructed label. The result supports explicit comparison with the fast action as an admission signal. The backend replacement, traffic sweep, and cross-simulator studies

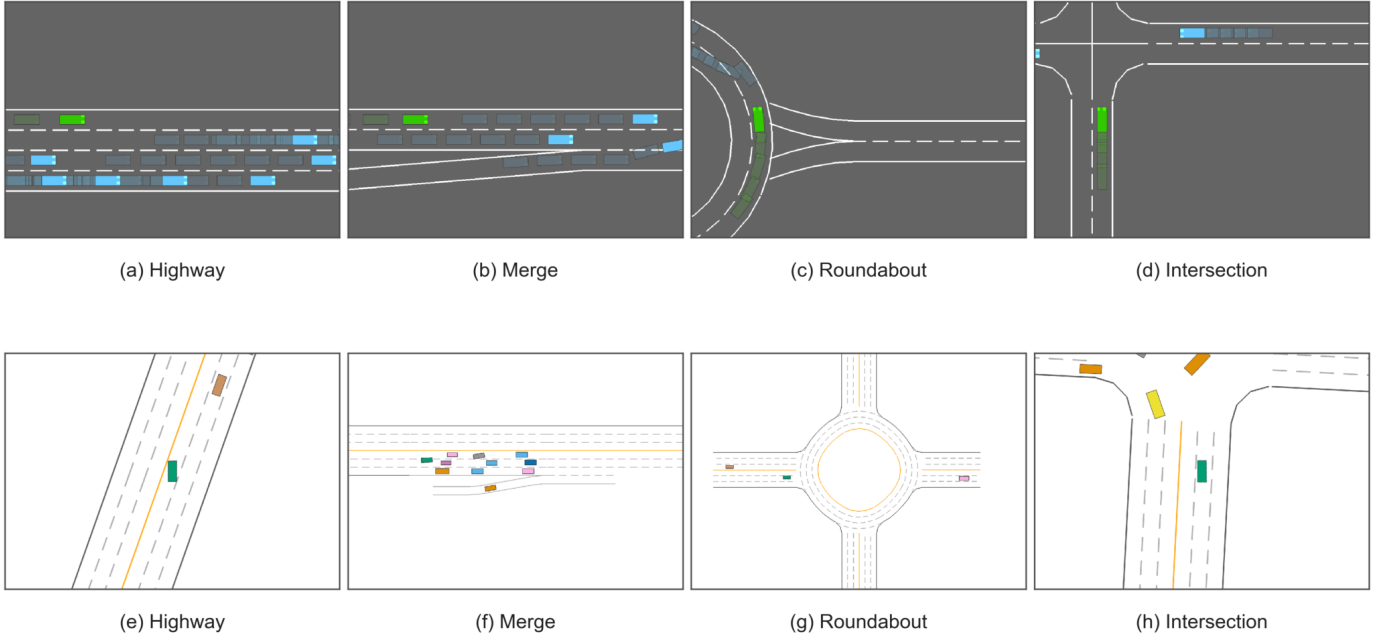


Fig. 6. Representative frozen closed-loop states under the shared RGD interface. The upper row shows highway-env: (a) highway, (b) merge, (c) roundabout, and (d) intersection. The lower row shows recorded-state reconstructions of the corresponding MetaDrive scenes: (e) highway, (f) merge, (g) roundabout, and (h) intersection. The ego vehicle is shown in green in every panel.

test a separate claim: the gate and release contract operate through a shared action interface rather than a reasoner-specific output format or a single road geometry. The present scope uses controlled added delays and discrete high-level actions; measured service latency and continuous trajectory proposals are natural extensions.

V. CONCLUSION

This paper presents Recoverability-Gated Deliberation, a two-stage framework that determines when to invoke slow reasoning and whether a delayed LLM proposal should enter the control loop. By separating query admission from release validation, RGD invokes slow reasoning only when the joint delay, support, cost, and demand conditions hold. It authorizes a returned proposal only when the mapped action is feasible, distinct from the fast action, and lower in recovery cost. R-VoD provides an offline matched-rollout measure of the corrective opportunity available at release.

Controlled-delay experiments, response tracing, and component analysis verify the complementary roles of query admission and release validation: a returned slow proposal changes the final command only after all release checks and the shared final projection, while the fast action remains in control otherwise. The same RGD interface operates across the evaluated LLM backends, traffic settings, and simulator adapters. By treating slow reasoning as an expiring proposal rather than a persistent command, RGD provides a practical release interface between low-rate deliberation and high-frequency control.

REFERENCES

- [1] L. Wen *et al.*, “DiLu: A knowledge-driven approach to autonomous driving with large language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [2] Z. Xu *et al.*, “DriveGPT4: Interpretable end-to-end autonomous driving via large language model,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 10, pp. 8186–8193, 2024.
- [3] H. Shao *et al.*, “LMDrive: Closed-loop end-to-end driving with large language models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 120–15 130.
- [4] C. Sima *et al.*, “DriveLM: Driving with graph visual question answering,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 256–274.
- [5] J. Mao, Y. Qian, J. Ye, H. Zhao, and Y. Wang, “GPT-Driver: Learning to drive with GPT,” arXiv preprint arXiv:2310.01415, 2023. [Online]. Available: <https://arxiv.org/abs/2310.01415>
- [6] Y. Hu, D. Ou, J. Huang, M. Wu, M. Hao, and R. Yu, “Integrating vision and language foundation models for enhanced navigation and decision-making in connected autonomous vehicles,” *IEEE Trans. Veh. Technol.*, vol. 74, no. 10, pp. 16 233–16 249, 2025.
- [7] D. Pei, J. He, K. Liu, Y. Wu, Y. Lei, and X. Xiao, “Enhancement of large language models driving knowledge for practical autonomous driving decision making,” *IEEE Trans. Veh. Technol.*, pp. 1–17, 2026, early access.
- [8] R. Xin *et al.*, “NetRoller: Interfacing general and specialized models for end-to-end autonomous driving,” *IEEE Trans. Veh. Technol.*, vol. 75, no. 5, pp. 7469–7482, 2026.
- [9] K. Qian *et al.*, “FASIONAD: Fast and slow fusion thinking systems for human-like autonomous driving with adaptive feedback,” arXiv preprint arXiv:2411.18013, 2024. [Online]. Available: <https://arxiv.org/abs/2411.18013>
- [10] R. Zhang *et al.*, “AdaDrive: Self-adaptive slow-fast system for language-grounded autonomous driving,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 5112–5121.
- [11] X. Chen, X. Wang, W. Chen, and J. Gao, “An adaptive multimodal end-to-end autonomous driving framework based on behavior discrepancy,” *IEEE Trans. Intell. Transp. Syst.*, pp. 1–12, 2026, early access.
- [12] B. Ma, H. Liu, R. Zhong, P. Liu, X. Zhou, and J. Ma, “Behavioral uncertainty-aware attention allocation via VLMs for interactive autonomous driving,” *IEEE Trans. Veh. Technol.*, pp. 1–14, 2026, early access.
- [13] Y. Chen, Z.-H. Ding, Z. Wang, Y. Wang, L. Zhang, and S. Liu, “Asynchronous large language model enhanced planner for autonomous driving,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 22–38.
- [14] L. Li, J. Fang, J. Xue, and C. Lv, “Vision-language model-enabled dual-system for autonomous driving in safety-critical transportation,” *IEEE Trans. Intell. Transp. Syst.*, pp. 1–12, 2026, early access.
- [15] L. Chen *et al.*, “Driving with LLMs: Fusing object-level vector modality

- for explainable autonomous driving,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 14 093–14 100.
- [16] R. Zhao *et al.*, “VLM-Driver: Human-like autonomous driving decision-making via vision language model,” *IEEE Trans. Veh. Technol.*, vol. 75, no. 5, pp. 7327–7342, 2026.
 - [17] J. Wang, H. Ren, X. Zhu, and Z. Ma, “Enhancing autonomous vehicle decision-making through policy transfer with large language model,” *IEEE Trans. Intell. Transp. Syst.*, pp. 1–10, 2025, early access.
 - [18] Z. Cui *et al.*, “C-TRAIL: A commonsense world framework for trajectory planning in autonomous driving,” *IEEE Trans. Veh. Technol.*, pp. 1–14, 2026, early access.
 - [19] X. Wang, Z. Zhu, G. Huang, X. Chen, J. Zhu, and J. Lu, “DriveDreamer: Towards real-world-drive world models for autonomous driving,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 55–72.
 - [20] C. Min *et al.*, “DriveWorld: 4D pre-trained scene understanding via world models for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 522–15 533.
 - [21] W. Zheng, R. Song, X. Guo, C. Zhang, and L. Chen, “GenAD: Generative end-to-end autonomous driving,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 87–104.
 - [22] B. Liao *et al.*, “DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 12 037–12 047.
 - [23] J. Mei *et al.*, “Continuously learning, adapting, and improving: A dual-process approach to autonomous driving,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 123 261–123 290.
 - [24] Y. Ma *et al.*, “LeapVAD: A leap in autonomous driving via cognitive perception and dual-process thinking,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 37, no. 4, pp. 1963–1977, 2026.
 - [25] Y. Luo *et al.*, “AdaThinkDrive: Adaptive thinking via reinforcement learning for autonomous driving,” arXiv preprint arXiv:2509.13769, 2025. [Online]. Available: <https://arxiv.org/abs/2509.13769>
 - [26] S. Russell and E. Wefald, “Principles of metareasoning,” *Artif. Intell.*, vol. 49, no. 1–3, pp. 361–395, 1991.
 - [27] M. Boddy and T. L. Dean, “Deliberation scheduling for problem solving in time-constrained environments,” *Artif. Intell.*, vol. 67, no. 2, pp. 245–285, 1994.
 - [28] S. Zilberstein, “Using anytime algorithms in intelligent systems,” *AI Mag.*, vol. 17, no. 3, pp. 73–83, 1996.
 - [29] B. Cserna, W. Ruml, and J. Frank, “Planning time to think: Metareasoning for on-line planning with durative actions,” in *Proc. Int. Conf. Autom. Plan. Scheduling (ICAPS)*, vol. 27, 2017, pp. 56–60.
 - [30] A. Elboher, A. Bensoussan, E. Karpas, W. Ruml, S. S. Shperberg, and E. Shimony, “A formal metareasoning model of concurrent planning and execution,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 10, 2023, pp. 12 427–12 435.
 - [31] P. Tabuada, “Event-triggered real-time scheduling of stabilizing control tasks,” *IEEE Trans. Autom. Control*, vol. 52, no. 9, pp. 1680–1685, 2007.
 - [32] M. Li, J. Gao, L. Zhao, and X. Shen, “Adaptive computing scheduling for edge-assisted autonomous driving,” *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5318–5331, 2021.
 - [33] T. G. Molnar, A. K. Kiss, A. D. Ames, and G. Orosz, “Safety-critical control with input delay in dynamic environment,” *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 4, pp. 1507–1520, 2023.
 - [34] S. Chen, Y. Sun, D. Li, Q. Wang, Q. Hao, and J. Sifakis, “Runtime safety assurance for learning-enabled control of autonomous driving vehicles,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 8978–8984.
 - [35] S. Shalev-Shwartz, S. Shammah, and A. Shashua, “On a formal model of safe and scalable self-driving cars,” arXiv preprint arXiv:1708.06374, 2017. [Online]. Available: <https://arxiv.org/abs/1708.06374>
 - [36] A. Liniger and J. Lygeros, “Real-time control for autonomous racing based on viability theory,” *IEEE Trans. Control Syst. Technol.*, vol. 27, no. 2, pp. 464–478, 2019.
 - [37] E. Leurent, “An environment for autonomous driving decision-making,” GitHub repository, 2018, version 1.12. [Online]. Available: <https://github.com/Farama-Foundation/HighwayEnv>
 - [38] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, “MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3461–3475, 2023.
 - [39] A. Yang *et al.*, “Qwen3 technical report,” arXiv preprint arXiv:2505.09388, 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
 - [40] G. Kou *et al.*, “PADriver: Towards personalized autonomous driving,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2025, pp. 1–8.